Midterm 1 – Tic Tac Toe

Varun Senthil Kumar, vsenth13@asu.edu, ASU ID: 1233679566

I. Introduction

The objective of this project is to design and program the Dobot to play a game of Tic Tac Toe against a human opponent. The system integrates computer vision, robotic control, and decision-making algorithms to enable real-time interaction between the player and the robot. A Dobot robotic arm is used to physically perform the moves on the game board, while a live camera feed is employed to detect and track the player's moves using color-based image processing techniques.

This project aims to demonstrate the integration of computer vision and robotic manipulation within a cohesive and autonomous system. The robot perceives the environment and makes strategic decisions based on the current game state, executing well-thought moves to place its tokens accordingly.

By the end of this project, the goal is to achieve a fully functional Tic Tac Toe robot that can visually detect the game grid, identify human moves accurately, decide optimal counter-moves using the minimax algorithm, detect incorrect moves and execute moves using the Dobot arm with minimal human intervention.

II. Procedure

All required packages and dependencies were installed on the Linux machine. Instead of the traditional tic-tac-toe game using pen and paper, this project was done using colored blocks, each representing X and O respectively, and the block pallet was used as the grid. We chose the red and yellow blocks for our game. Using knowledge from Lab 2, we used the suction cup end effector for the Dobot to pick and place the blocks.

Using knowledge from Lab 3, we mounted the camera above the Dobot end effector. Multiple python scripts were used to calibrate the camera and test OpenCV-based color detection, grid detection and grid mapping. This allowed us to ensure that the camera detected the colored blocks accurately, the pallet as the grid, and mapping of the pallet to suit the game.

Before the program starts, both sets of blocks are stacked on either side of the pallet. When the program starts, the player selects who plays first (robot/human) and which color (red or yellow) they will use. The robot is automatically assigned the opposite color and corresponding source position for the blocks on the workspace. Figure 1 shows the setup for the project.

The camera feed initializes with a visible 3×3 grid overlay that maps the board's physical coordinates to digital grid indices. During gameplay, OpenCV continuously captures frames from the camera. HSV color filtering identifies red or yellow blocks placed by the player. The detected block centers are converted into grid positions, updating the game board matrix.

The program checks for invalid moves, including using the wrong color, placing multiple blocks at once, or selecting an already occupied cell. Each move must remain stable in the same position for a few frames before being accepted, preventing false detections due to lighting or camera noise.

After a valid human move, the program evaluates the game state using the Minimax algorithm, which simulates all possible outcomes to choose the robot's optimal move. The robot's movement coordinates are pre-defined for each grid cell.

During its turn, the robot moves from the home position to its source stack (red or yellow), picks up a block using the suction cup, and places it precisely in the corresponding board position. After each move, the robot returns to the home position to ensure consistent motion paths and avoid interference with the camera view.

The program continues alternating turns between the player and the robot until a win or draw condition is reached. When the game ends, a message ("Human Wins," "Robot Wins," or "Draw") is displayed on the live camera feed for a few seconds, allowing the user to see the final state of the board before the program closes.

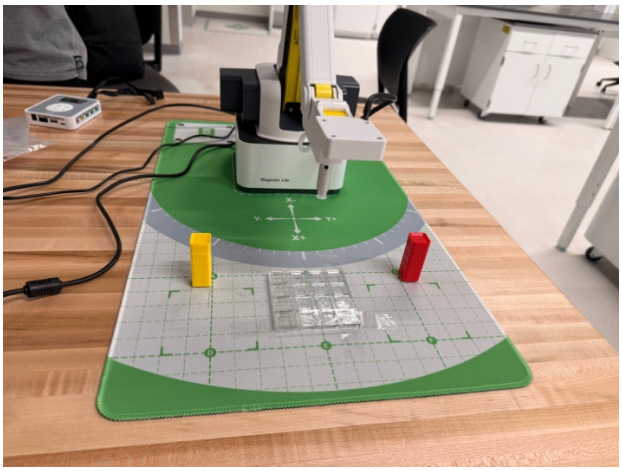

Figure 1: The Dobot, Pallet and block setup

Table 1 contains the coordinates for the blocks' source positions, while Table 2 contains the coordinates for the positions on the grid.

Table 1 : Source position coordinates

	x	y	z
Red blocks source position	255	104	-6
Yellow blocks source position	255	-112	-6

Since the blocks are stacked on top of one another, the program decreases the source position's height by 10mm every time a block is picked, to make it autonomous.

Table 2: Grid Position Coordinates

[0,0] – (295,5,-38)	[0,1] – (295,-15,-38)	[0,2] – (295,-35,-38)
[1,0] – (275,5,-38)	[1,1] – (275,-15,-38)	[1,2] – (275,-35,-38)
[2,0] – (255,5,-38)	[2,1] – (255,-15,-38)	[2,2] – (255,-35,-38)

III. RESULTS

The developed Tic Tac Toe robot program successfully integrated real-time vision, game logic, and robotic control. The system was able to detect the player's move accurately using color-based recognition through the camera feed. The grid calibration and pixel-to-grid mapping worked effectively, allowing the program to interpret the player's moves correctly and respond with corresponding robotic actions.

The robotic arm executed pick-and-place operations reliably from the pre-defined color source positions to the correct grid cells. The live camera feed remained responsive throughout gameplay, displaying the grid overlay and current game status. The program correctly identified win and draw conditions and displayed the corresponding messages at the end of each game.

However, while the system functioned as intended in terms of move detection and robotic control, the robot did not always select the best possible move according to the game's Minimax algorithm. In some test runs, the robot made suboptimal moves, occasionally allowing the human player to win despite having potential blocking or winning moves available. This behavior suggests that the AI's decision-making logic or symbol assignment may need further refinement to ensure consistent optimal move selection regardless of which color the robot is assigned.

Overall, the system demonstrated successful operation in detecting moves, handling gameplay flow, and controlling the robotic arm, but improvements are required in the AI move evaluation to achieve fully optimal play.

IV. DISCUSSION

While the Tic Tac Toe system operated successfully in terms of visual detection, move validation, and robotic control, the robot's gameplay behavior revealed inconsistencies in its decision-making process. The main issue identified was related to the symbol assignment within the Minimax algorithm. In the initial version, the Minimax function was hard coded to assume the Dobot always used the same symbol (specifically symbol "O"), as the color choice option was introduced later.

This static assignment caused the algorithm to evaluate the board from the wrong perspective when the Dobot was assigned symbol "X." As a result, the scoring function favored the human player's potential outcomes, leading the Dobot to make suboptimal moves or fail to block obvious winning opportunities. The system's decision logic was therefore inconsistent with the expected optimal play behavior of the Minimax algorithm.

To address this issue, the Minimax and evaluation functions were updated to dynamically accept both the Dobot's symbol and the human's symbol as parameters. This modification ensures that the algorithm correctly identifies which player is maximizing (the Dobot) and which is minimizing (the human), allowing move evaluations to align with the Dobot's actual color and role in the game.

However, due to time constraints, the corrected version of the algorithm could not be fully tested on the physical robot. Thus, while the theoretical fix resolves the logical inconsistency in the AI's design, its performance under real gameplay conditions remains unverified. Future testing should focus on validating the updated algorithm's decision-making behavior and tuning the evaluation function to further enhance move selection efficiency and strategic accuracy.

V. BONUS

The bonus part of the midterm was also incorporated into the program. The program successfully identified invalid moves, such as placing incorrect color block, placing multiple blocks simultaneously or attempting to occupy an already filled position.

The images below show the error handling messages thrown by the program when invalid moves are detected.

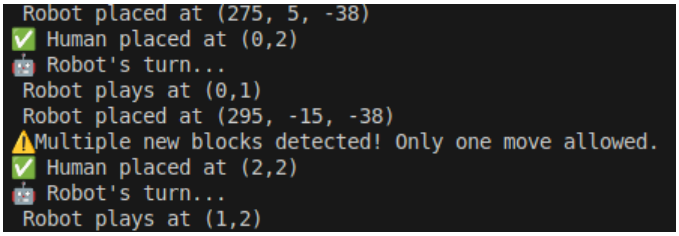

Figure 2 : Error message for multiple blocks edge case

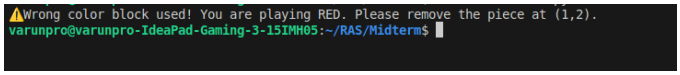

Figure 3 : Error message for incorrect color block edge case

VI. Technical Specifications

Robot Name: Dobot Magician Lite
Robot Serial Number: DT-MGL-4R002-01E
Repeatability: ±0.2mm
Maximum Payload: 0.25kg
Maximum Reach: 340mm
End Effector: Suction Cup , Camera
Software Platform: Python
Safety Features: Collision Detection

VII. Video and Code

Code Repository:
https://github.com/VarunPro23/RAS/tree/main/Midterm

- *XO_main.py* – The main program script, contains all game logic, Dobot movement and camera.
- *color_test.py* – Python script to check whether the camera detects the block's color accurately.
- *block_test.py* – Python script to check whether the Dobot does the pick-and-place movement accurately, and to determine the coordinates of the grid positions.
- *position_test.py* – Python script to check whether OpenCV detects and maps the grid accurately.
- *result.py* – Python script to check whether the program and camera detects win/draw scenarios.
- *edgecases.py* – Python script to check whether the program and camera detects the edge case scenarios.

YouTube URL:
https://youtu.be/nvik8Fx5WHw